Websocket Endpoint Vulnerability Analysis Report

Testing Insecure WebSocket Implementations: Crawling Public Sites, Testing Endpoints for Vulnerabilities, and Reporting Impact Analysis

Report Generated on : July 01, 2026

PESU Center for
Information Security,
Forensics and Cyber Resilience

PESU RESEARCH
11TH FLOOR, BE BLOCK,
PES UNIVERSITY, RING ROAD CAMPUS,
BANGALORE, INDIA

WebSocket Security Scan Report

Executive Summary

Real-time apps increasingly rely on WebSocket connections, but insecure implementations—such as missing origin checks or weak authentication—can allow hijacking or sensitive data exposure.

To address this, we developed an automated scanner that crawls public web applications, detects vulnerable WebSocket endpoints, and analyzes potential security threats. The program does the following:

- Crawl and detect active WebSocket endpoints from public websites.
- Apply 90 specialized tests for WebSockets to assess security gaps.
- Generate a structured PDF report summarizing detected vulnerabilities and severity.

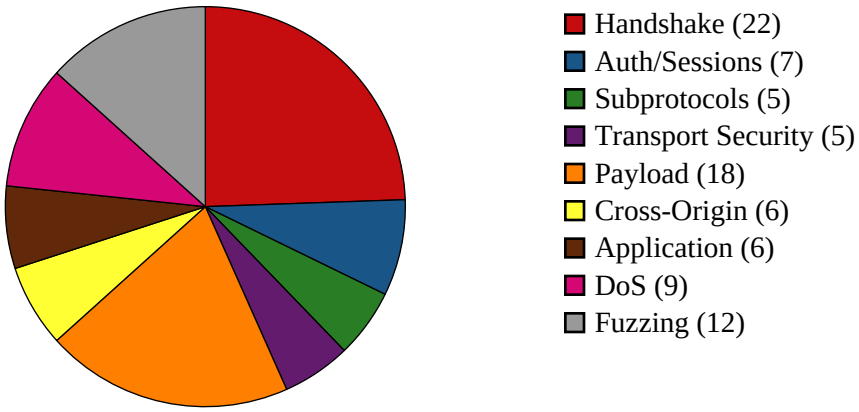
Scan Start Time:	2026-07-01 17:00:06
Scan End Time:	2026-07-01 17:06:10
Total Scan Duration:	364.46 seconds
Total URLs Scanned:	0
High Severity Vulnerabilities:	132
Medium Severity Vulnerabilities:	69
Low Severity Vulnerabilities:	29

Quick Insights

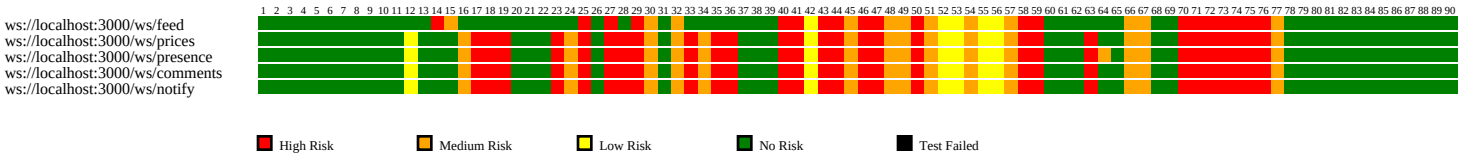
This section lists all scanned websites, shows the overall vulnerability test distribution by category with a pie chart, displays a heatmap which shows how the WebSockets of a website responded to the attacks, and shows the overall count of vulnerabilities found by category in a tabular form and through a bar chart. These quick insights can be used for analysis at a glance and understanding patterns in security issues easily.

#	Website

Test Type Distribution

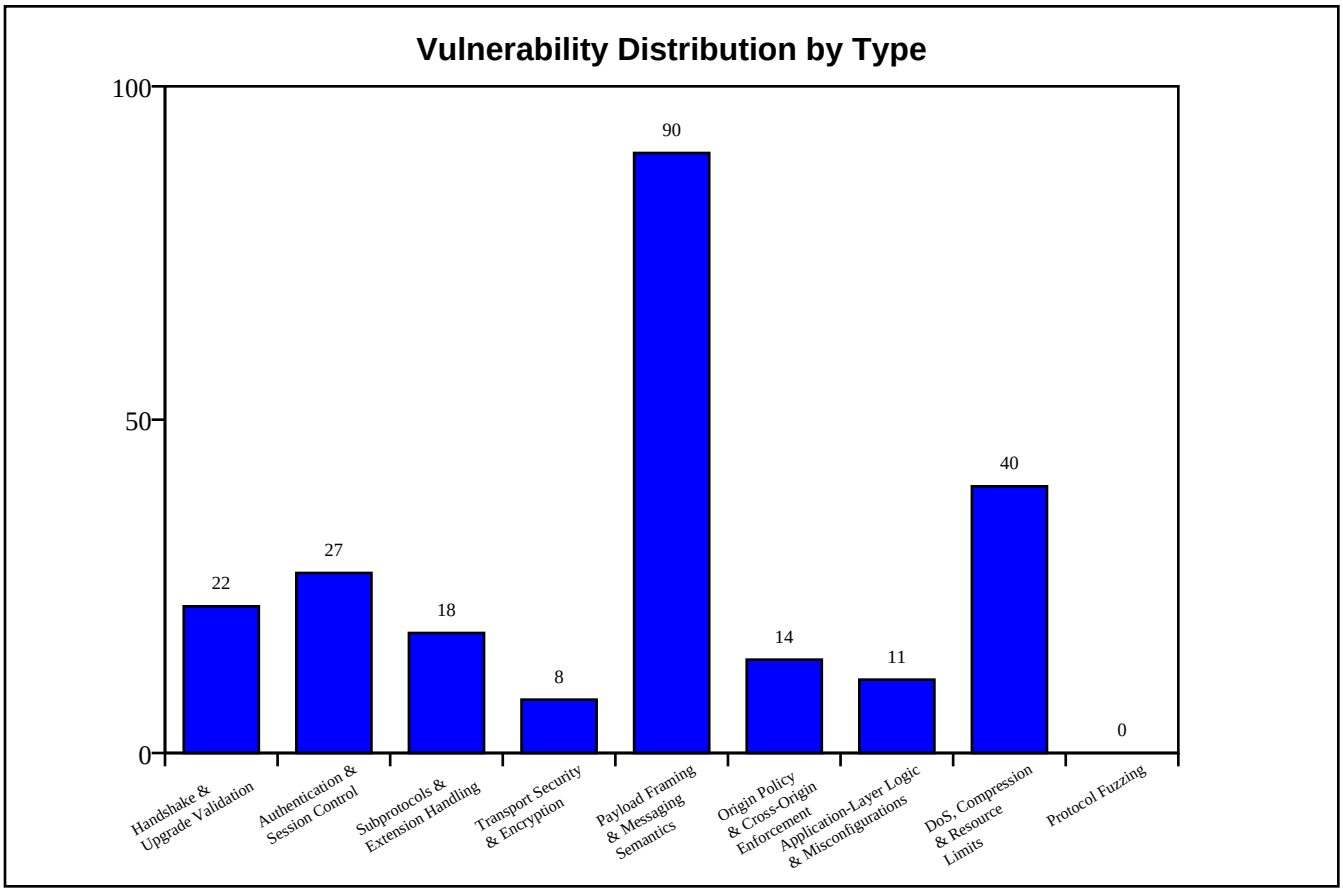


WebSocket vs. Attack Heatmap



Vulnerability Summary by Type

Type	Count
Handshake & Upgrade Validation	22
Authentication & Session Control	27
Subprotocols & Extension Handling	18
Transport Security & Encryption	8
Payload Framing & Messaging Semantics	90
Origin Policy & Cross-Origin Enforcement	14
Application-Layer Logic & Misconfigurations	11
DoS, Compression & Resource Limits	40
Protocol Fuzzing	0


Vulnerability Distribution by Type
100
50
0
22
27
18
8
90
14
11
40
0
Handshake & Upgrade Validation
Authentication & Session Control
Subprotocols & Extension Handling
Transport Security & Encryption
Payload Framing & Messaging Semantics
Origin Policy & Cross-Origin Enforcement
Application-Layer Logic & Misconfigurations
DoS, Compression & Resource Limits
Protocol Fuzzing

Detailed Scan Results

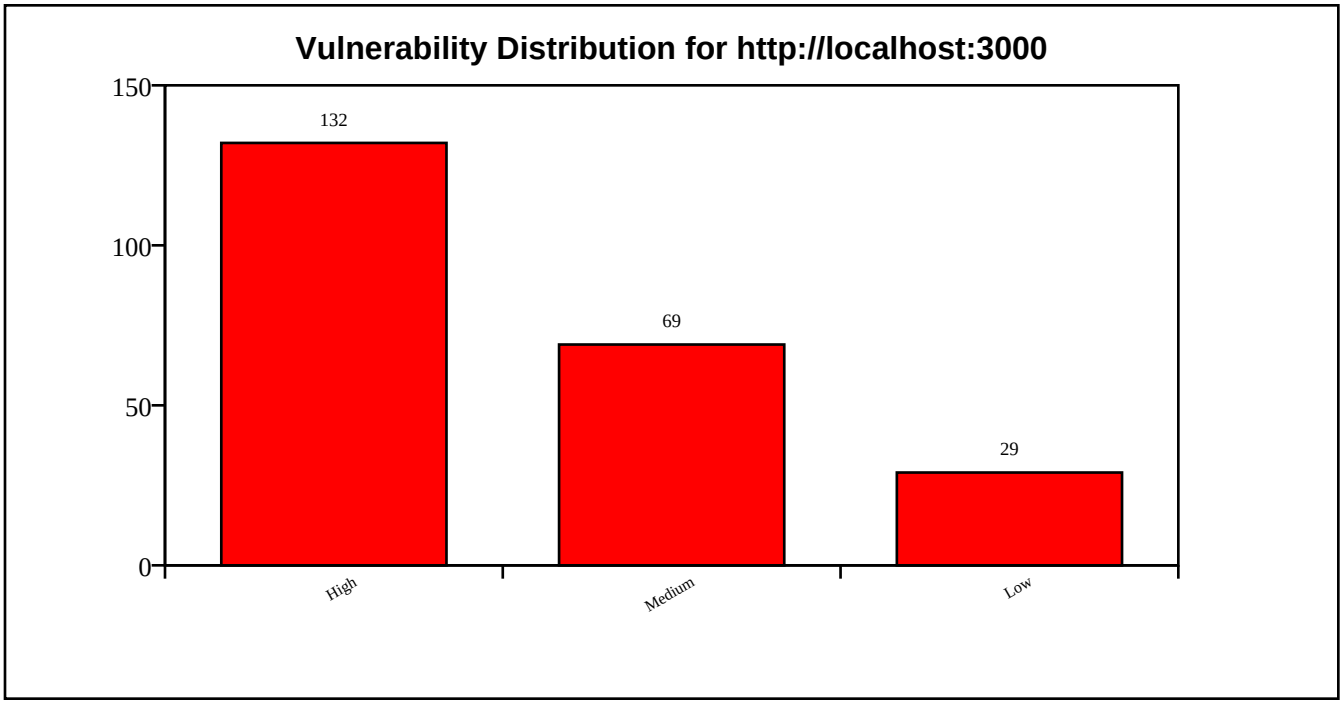
This section provides an in-depth breakdown of each scanned URL. For every URL, it lists the scan duration, number of URLs crawled, and the number of WebSocket endpoints discovered. It helps identify how many potential communication channels were exposed for testing. Each target's vulnerability distribution is summarized by severity (High, Medium, Low) in the table and by a bar chart, followed by a detailed list of detected vulnerabilities. This allows for a thorough understanding of the security posture and exposure of each target URL. It is to be noted that all WebSocket Endpoints are publicly listed and there is no intent to attack or damage servers.

Target URL: http://localhost:3000

Scan Duration:	360.17 seconds
WebSocket Endpoints Found via Crawling:	5
Attack Performed:	True
High Severity Findings:	132
Medium Severity Findings:	69
Low Severity Findings:	29

WebSocket Endpoints:

#	URL
1	ws://localhost:3000/ws/feed
2	ws://localhost:3000/ws/prices
3	ws://localhost:3000/ws/presence
4	ws://localhost:3000/ws/comments
5	ws://localhost:3000/ws/notify



Detected Vulnerabilities:

This section lists all vulnerabilities identified during the scan of the target URL. Each entry includes the vulnerability name, its severity (High, Medium, or Low), a description of the issue, and recommended solutions. This detailed information helps understand the exact flaws present in the WebSocket implementation of each target.

Affected WebSocket Endpoint: ws://localhost:3000/ws/presence

Fake HTTP Status

Risk Level: High

Description: Server at localhost:3000 returned unexpected status: HTTP/1.1 403 Forbidden

Solution: Ensure server returns "HTTP/1.1 101 Switching Protocols" for valid handshakes.

Wrong Sec-WebSocket-Accept

Risk Level: Medium

Description: Server at localhost:3000 did not return a Sec-WebSocket-Accept header.

Solution: Ensure server follows RFC 6455 and sends correct Sec-WebSocket-Accept header.

Fake Token

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepts connections with a fake authentication token.

Solution: Implement robust token validation (e.g., JWT signature verification, token expiry check, audience validation).

Stale Session Reconnect

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence allows reconnection with same stale session cookie.

Solution: Invalidate old session IDs on WebSocket reconnect. Require fresh authentication or refresh token.

Missing Authentication

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence allows unauthenticated connections and responds with data.

Solution: Require authentication (e.g., JWT, API keys) for WebSocket connections.

Invalid Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence negotiated invalid subprotocol: 'invalid..protocol'.

Solution: Reject malformed or unsupported subprotocol values during handshake.

Unaccepted Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence negotiated unadvertised subprotocol 'unadvertised_protocol'.

Solution: Only negotiate subprotocols explicitly supported by the server.

Undefined Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted frame with undefined opcode 0x3.

Solution: Reject frames with undefined opcodes.

Reserved Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted frame with reserved opcode 0xB.

Solution: Reject frames with reserved opcodes (0x3-0x7, 0xB-0xF).

Zero-Length Fragment

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/presence accepted zero-length fragments and responded unexpectedly.

Solution: Reject or limit incomplete fragmented messages.

Invalid Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted frame with declared payload length 10 but

sent only 4 bytes.

Solution: Validate payload length matches actual data.

Negative Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted forged extended payload length (0x8000000000000001).

Solution: Validate payload length fields and reject extreme or invalid values.

Mismatched Payload

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence accepted frames with mismatched lengths.

Solution: Ensure payload lengths match.

Invalid Masking Key

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted a frame with invalid masking key pattern: All-zero.

Solution: Enforce strict validation of client masking keys per RFC 6455.

Unmasked Client Frame

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted an unmasked client frame.

Solution: Require masking for all client-to-server frames per RFC 6455.

Invalid RSV Bits

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence accepted a frame with invalid RSV1 bit set.

Solution: Reject non-zero RSV bits unless explicitly negotiated via extension.

Oversized Control Frame

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence accepted a ping control frame with 126-byte payload.

Solution: Reject control frames larger than 125 bytes as per RFC 6455.

Non-UTF-8 Text

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted a text frame with invalid UTF-8 bytes.

Solution: Ensure strict UTF-8 validation of text frames.

Null Bytes in Text

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence accepted a text frame containing null bytes.

Solution: Validate and sanitize text frames for embedded nulls. Avoid C-style string truncation risks.

Binary as Text

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/presence accepted a text frame with non-UTF-8 binary data.

Solution: Validate UTF-8 compliance in all text frames as per RFC 6455.

Text as Binary

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/presence accepted UTF-8 text sent in a binary frame.

Solution: Handle binary and text frames with separate logic as per RFC 6455.

Invalid Close Code

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence accepted a close frame with invalid code 999.

Solution: Close codes must conform to RFC 6455 (valid: 1000-1015, 3000-4999).

Early Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/presence accepted an early close frame before any data was exchanged.

Solution: Gracefully handle close frames sent immediately after handshake.

No Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/presence handled abrupt TCP closure and allowed clean reconnection.

Solution: Ensure that server detects and cleans up on ungraceful disconnects.

Long Close Reason

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence accepted close frame with long reason (123 bytes).

Solution: Enforce strict limits on close reason size (≤123 bytes).

Missing CORS Headers

Risk Level: High

Description: WebSocket endpoint ws://localhost:3000/ws/presence (HTTP equivalent) lacks proper CORS headers.

Solution: Implement proper CORS headers to restrict cross-origin access.

Cross-Origin Iframe

Risk Level: High

Description: ws://localhost:3000/ws/presence allows itself to be embedded in cross-origin iframes (missing X-Frame-Options / CSP).

Solution: Set X-Frame-Options: DENY or SAMEORIGIN, or CSP frame-ancestors directive.

Invalid Content-Type

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/presence (HTTP equivalent) serves invalid Content-Type: text/html.

Solution: Ensure WebSocket endpoints return appropriate Content-Type or upgrade headers.

Missing Security Headers

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/presence (HTTP equivalent) lacks the following headers: Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options.

Solution: Add missing security headers such as Content-Security-Policy, X-Frame-Options, and

Strict-Transport-Security.

Connection Flood

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence allowed 100 concurrent connections in 1.35s.

Solution: Enforce per-IP connection limits and rate limiting to prevent abuse.

Oversized Message

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted a 10MB message.

Solution: Set a reasonable max message size limit (e.g., 1MB) to prevent buffer overflows.

Max Connections

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence allows 100 simultaneous connections without restriction.

Solution: Enforce a maximum connection limit per client to prevent resource exhaustion.

Idle Timeout Abuse

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence allows idle connections to persist for 60 seconds.

Solution: Implement an idle timeout policy to close inactive connections.

High Compression Ratio

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepts highly compressible messages (1MB of 'A').

Solution: Limit allowed compression ratio or message size on the server.

Large Payload Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted repeated large messages without closing.

Solution: Set server-side limits for message size and rate. Monitor memory usage.

TCP Half-Open Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/presence accepted hanging TCP connections without timeout.

Solution: Use TCP keep-alive and server-side timeout policies.

No Compression Negotiation

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/presence may mishandle compression without proper negotiation.

Solution: Ensure the server only decompresses messages when permessage-deflate was negotiated.

Affected WebSocket Endpoint: ws://localhost:3000/ws/feed

Case-Sensitive Headers

Risk Level: Low

Description: Server at localhost:3000 accepted case-sensitive headers.

Solution: Ensure case-insensitive header parsing as per RFC.

Oversized Headers

Risk Level: Medium

Description: Server at localhost:3000 accepted handshake with oversized headers.

Solution: Set limits for header size to prevent resource exhaustion.

Missing Host Header

Risk Level: High

Description: Server at localhost:3000 accepted handshake without Host header.

Solution: Enforce Host header validation.

Fake Host Header

Risk Level: High

Description: Server at localhost:3000 accepted handshake with incorrect Host header.

Solution: Validate Host header to match expected server domain.

Multiple Host Headers

Risk Level: High

Description: Server at localhost:3000 accepted handshake with multiple Host headers.

Solution: Reject requests with duplicate Host headers.

No Session Cookie

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepts connections without a session cookie.

Solution: Require valid session cookies (or tokens) to authenticate WebSocket clients.

Expired Cookie

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed accepts connections with an expired session cookie.

Solution: Validate cookie expiration on the server side and reject expired tokens.

Fake Token

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepts connections with a fake authentication token.

Solution: Implement robust token validation (e.g., JWT signature verification, token expiry check, audience validation).

Stale Session Reconnect

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed allows reconnection with same stale session cookie.

Solution: Invalidate old session IDs on WebSocket reconnect. Require fresh authentication or refresh token.

Cross-Site Cookie Hijack

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted cross-origin cookies and origin header.

Solution: Set SameSite=Strict on cookies and validate the Origin header server-side.

Missing Authentication

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed allows unauthenticated connections and responds with data.

Solution: Require authentication (e.g., JWT, API keys) for WebSocket connections.

Invalid Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed negotiated invalid subprotocol: 'invalid..protocol'.

Solution: Reject malformed or unsupported subprotocol values during handshake.

Unaccepted Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed negotiated unadvertised subprotocol 'unadvertised_protocol'.

Solution: Only negotiate subprotocols explicitly supported by the server.

Fake Extension

Risk Level: High

Description: Server at localhost:3000 accepted spoofed extension.

Solution: Validate Sec-WebSocket-Extensions header against supported values.

Conflicting Extensions

Risk Level: Medium

Description: Server at localhost:3000 accepted conflicting extensions.

Solution: Reject requests with duplicate or conflicting extensions.

Spoofed Connection Header

Risk Level: High

Description: Server at localhost:3000 accepted spoofed Connection header.

Solution: Strictly validate Connection header to be exactly "Upgrade".

HTTP/1.0 Downgrade

Risk Level: High

Description: Server at localhost:3000 accepted HTTP/1.0 WebSocket handshake.

Solution: Only allow WebSocket upgrades over HTTP/1.1 or newer.

Undefined Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted frame with undefined opcode 0x3.

Solution: Reject frames with undefined opcodes.

Reserved Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted frame with reserved opcode 0xB.

Solution: Reject frames with reserved opcodes (0x3-0x7, 0xB-0xF).

Zero-Length Fragment

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/feed accepted zero-length fragments and responded unexpectedly.

Solution: Reject or limit incomplete fragmented messages.

Invalid Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted frame with declared payload length 10 but sent only 4 bytes.

Solution: Validate payload length matches actual data.

Negative Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted forged extended payload length (0x8000000000000001).

Solution: Validate payload length fields and reject extreme or invalid values.

Mismatched Payload

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed accepted frames with mismatched lengths.

Solution: Ensure payload lengths match.

Invalid Masking Key

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted a frame with invalid masking key pattern: All-zero.

Solution: Enforce strict validation of client masking keys per RFC 6455.

Unmasked Client Frame

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted an unmasked client frame.

Solution: Require masking for all client-to-server frames per RFC 6455.

Invalid RSV Bits

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed accepted a frame with invalid RSV1 bit set.

Solution: Reject non-zero RSV bits unless explicitly negotiated via extension.

Oversized Control Frame

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed accepted a ping control frame with 126-byte payload.

Solution: Reject control frames larger than 125 bytes as per RFC 6455.

Non-UTF-8 Text

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted a text frame with invalid UTF-8 bytes.

Solution: Ensure strict UTF-8 validation of text frames.

Null Bytes in Text

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed accepted a text frame containing null bytes.

Solution: Validate and sanitize text frames for embedded nulls. Avoid C-style string truncation risks.

Binary as Text

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/feed accepted a text frame with non-UTF-8 binary data.

Solution: Validate UTF-8 compliance in all text frames as per RFC 6455.

Text as Binary

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/feed accepted UTF-8 text sent in a binary frame.

Solution: Handle binary and text frames with separate logic as per RFC 6455.

Invalid Close Code

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed accepted a close frame with invalid code 999.

Solution: Close codes must conform to RFC 6455 (valid: 1000-1015, 3000-4999).

Early Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/feed accepted an early close frame before any data was exchanged.

Solution: Gracefully handle close frames sent immediately after handshake.

No Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/feed handled abrupt TCP closure and allowed clean reconnection.

Solution: Ensure that server detects and cleans up on ungraceful disconnects.

Long Close Reason

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed accepted close frame with long reason (123 bytes).

Solution: Enforce strict limits on close reason size (≤123 bytes).

Missing CORS Headers

Risk Level: High

Description: WebSocket endpoint ws://localhost:3000/ws/feed (HTTP equivalent) lacks proper CORS headers.

Solution: Implement proper CORS headers to restrict cross-origin access.

Cross-Origin Iframe

Risk Level: High

Description: ws://localhost:3000/ws/feed allows itself to be embedded in cross-origin iframes (missing X-Frame-Options / CSP).

Solution: Set X-Frame-Options: DENY or SAMEORIGIN, or CSP frame-ancestors directive.

Missing Origin Check

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepts connections from unauthorized origin 'http://malicious-site.com'.

Solution: Implement strict Origin header validation (whitelist allowed domains).

Invalid Content-Type

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/feed (HTTP equivalent) serves invalid Content-Type: text/html.

Solution: Ensure WebSocket endpoints return appropriate Content-Type or upgrade headers.

Missing Security Headers

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/feed (HTTP equivalent) lacks the following headers: Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options.

Solution: Add missing security headers such as Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security.

Connection Flood

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed allowed 100 concurrent connections in 1.78s.

Solution: Enforce per-IP connection limits and rate limiting to prevent abuse.

Oversized Message

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted a 10MB message.

Solution: Set a reasonable max message size limit (e.g., 1MB) to prevent buffer overflows.

Max Connections

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed allows 100 simultaneous connections without restriction.

Solution: Enforce a maximum connection limit per client to prevent resource exhaustion.

Idle Timeout Abuse

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed allows idle connections to persist for 60 seconds.

Solution: Implement an idle timeout policy to close inactive connections.

High Compression Ratio

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepts highly compressible messages (1MB of 'A').

Solution: Limit allowed compression ratio or message size on the server.

Large Payload Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted repeated large messages without closing.

Solution: Set server-side limits for message size and rate. Monitor memory usage.

TCP Half-Open Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/feed accepted hanging TCP connections without timeout.

Solution: Use TCP keep-alive and server-side timeout policies.

No Compression Negotiation

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/feed may mishandle compression without proper negotiation.

Solution: Ensure the server only decompresses messages when permessage-deflate was negotiated.

Affected WebSocket Endpoint: ws://localhost:3000/ws/prices

Case-Sensitive Headers

Risk Level: Low

Description: Server at localhost:3000 accepted case-sensitive headers.

Solution: Ensure case-insensitive header parsing as per RFC.

Oversized Headers

Risk Level: Medium

Description: Server at localhost:3000 accepted handshake with oversized headers.

Solution: Set limits for header size to prevent resource exhaustion.

Missing Host Header

Risk Level: High

Description: Server at localhost:3000 accepted handshake without Host header.

Solution: Enforce Host header validation.

Fake Host Header

Risk Level: High

Description: Server at localhost:3000 accepted handshake with incorrect Host header.

Solution: Validate Host header to match expected server domain.

Multiple Host Headers

Risk Level: High

Description: Server at localhost:3000 accepted handshake with multiple Host headers.

Solution: Reject requests with duplicate Host headers.

No Session Cookie

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepts connections without a session cookie.

Solution: Require valid session cookies (or tokens) to authenticate WebSocket clients.

Expired Cookie

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices accepts connections with an expired session cookie.

Solution: Validate cookie expiration on the server side and reject expired tokens.

Fake Token

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepts connections with a fake authentication token.

Solution: Implement robust token validation (e.g., JWT signature verification, token expiry check, audience validation).

Stale Session Reconnect

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices allows reconnection with same stale session cookie.

Solution: Invalidate old session IDs on WebSocket reconnect. Require fresh authentication or refresh token.

Cross-Site Cookie Hijack

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted cross-origin cookies and origin header.

Solution: Set SameSite=Strict on cookies and validate the Origin header server-side.

Missing Authentication

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices allows unauthenticated connections and responds with data.

Solution: Require authentication (e.g., JWT, API keys) for WebSocket connections.

Invalid Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices negotiated invalid subprotocol: 'invalid..protocol'.

Solution: Reject malformed or unsupported subprotocol values during handshake.

Unaccepted Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices negotiated unadvertised subprotocol 'unadvertised_protocol'.

Solution: Only negotiate subprotocols explicitly supported by the server.

Fake Extension

Risk Level: High

Description: Server at localhost:3000 accepted spoofed extension.

Solution: Validate Sec-WebSocket-Extensions header against supported values.

Conflicting Extensions

Risk Level: Medium

Description: Server at localhost:3000 accepted conflicting extensions.

Solution: Reject requests with duplicate or conflicting extensions.

Spoofed Connection Header

Risk Level: High

Description: Server at localhost:3000 accepted spoofed Connection header.

Solution: Strictly validate Connection header to be exactly "Upgrade".

HTTP/1.0 Downgrade

Risk Level: High

Description: Server at localhost:3000 accepted HTTP/1.0 WebSocket handshake.

Solution: Only allow WebSocket upgrades over HTTP/1.1 or newer.

Undefined Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted frame with undefined opcode 0x3.

Solution: Reject frames with undefined opcodes.

Reserved Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted frame with reserved opcode 0xB.

Solution: Reject frames with reserved opcodes (0x3-0x7, 0xB-0xF).

Zero-Length Fragment

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/prices accepted zero-length fragments and responded unexpectedly.

Solution: Reject or limit incomplete fragmented messages.

Invalid Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted frame with declared payload length 10 but sent only 4 bytes.

Solution: Validate payload length matches actual data.

Negative Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted forged extended payload length (0x8000000000000001).

Solution: Validate payload length fields and reject extreme or invalid values.

Mismatched Payload

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices accepted frames with mismatched lengths.

Solution: Ensure payload lengths match.

Invalid Masking Key

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted a frame with invalid masking key pattern: All-zero.

Solution: Enforce strict validation of client masking keys per RFC 6455.

Unmasked Client Frame

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted an unmasked client frame.

Solution: Require masking for all client-to-server frames per RFC 6455.

Invalid RSV Bits

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices accepted a frame with invalid RSV1 bit set.

Solution: Reject non-zero RSV bits unless explicitly negotiated via extension.

Oversized Control Frame

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices accepted a ping control frame with 126-byte payload.

Solution: Reject control frames larger than 125 bytes as per RFC 6455.

Non-UTF-8 Text

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted a text frame with invalid UTF-8 bytes.

Solution: Ensure strict UTF-8 validation of text frames.

Null Bytes in Text

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices accepted a text frame containing null bytes.

Solution: Validate and sanitize text frames for embedded nulls. Avoid C-style string truncation risks.

Binary as Text

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/prices accepted a text frame with non-UTF-8 binary data.

Solution: Validate UTF-8 compliance in all text frames as per RFC 6455.

Text as Binary

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/prices accepted UTF-8 text sent in a binary frame.

Solution: Handle binary and text frames with separate logic as per RFC 6455.

Invalid Close Code

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices accepted a close frame with invalid code 999.

Solution: Close codes must conform to RFC 6455 (valid: 1000-1015, 3000-4999).

Early Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/prices accepted an early close frame before any data was exchanged.

Solution: Gracefully handle close frames sent immediately after handshake.

No Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/prices handled abrupt TCP closure and allowed clean reconnection.

Solution: Ensure that server detects and cleans up on ungraceful disconnects.

Long Close Reason

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices accepted close frame with long reason (123 bytes).

Solution: Enforce strict limits on close reason size (≤123 bytes).

Missing CORS Headers

Risk Level: High

Description: WebSocket endpoint ws://localhost:3000/ws/prices (HTTP equivalent) lacks proper CORS headers.

Solution: Implement proper CORS headers to restrict cross-origin access.

Cross-Origin Iframe

Risk Level: High

Description: ws://localhost:3000/ws/prices allows itself to be embedded in cross-origin iframes (missing X-Frame-Options / CSP).

Solution: Set X-Frame-Options: DENY or SAMEORIGIN, or CSP frame-ancestors directive.

Missing Origin Check

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepts connections from unauthorized origin 'http://malicious-site.com'.

Solution: Implement strict Origin header validation (whitelist allowed domains).

Error Message Leak

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices leaks sensitive error messages: {"type":"welcome","prices":{"BTC":62476.85,"ETH":3189.25,"SOL":147.39,"NEXUS":4.21},"env":{"ALLUSERSPROFILE":"C:\\ProgramData","APPDATA":"C:\\Users\\Tejas\\AppData\\Roaming","ChocolateyInstall":"C:\\ProgramData\\chocolatey","ChocolateyLastPathUpdate":"133702444167870130","CHROME_CRASHPAD_PIPE_NAME":"\\\\.\\pipe\\crashpad_4436_MWWSCEFIAZYCGIQG","CommonProgramFiles":"C:\\Program Files\\Common Files","CommonProgramFiles(x86)":"C:\\Program Files (x86)\\Common Files","CommonProgramW6432":"C:\\Program Files\\Common Files","COMPUTERNAME":"LAPTOP-KGMNJSJJ","ComSpec":"C:\\WINDOWS\\system32\\cmd.exe","DriverData":"C:\\Windows\\System32\\Drivers\\DriverData","EFC_19864_1592913036":"1","EFC_19864_4126798990":"1","HOMEDRIVE":"C:","HOMEPATH":"\\Users\\Tejas","LOCALAPPDATA":"C:\\Users\\Tejas\\AppData\\Local","LOGONSERVER":"\\\\LAPTOP-KGMNJSJJ","NUMBER_OF_PROCESSORS":"8","OneDrive":"D:\\OneDrive","OneDriveConsumer":"D:\\OneDrive","OnlineServices":"Online Services","OS":"Windows_NT","Path":"c:\\Users\\Tejas\\AppData\\Roaming\\Code\\User\\globalStorage\\github.copilot-chat\\debugCommand;c:\\Users\\Tejas\\AppData\\Roaming\\Code\\User\\globalStorage\\github.copilot-chat\\copilotCli;C:\\Program Files\\Common Files\\Oracle\\Java\\javapath;C:\\Python312\\Scripts\\;C:\\Python312\\;C:\\WINDOWS\\system32;C:\\WINDOWS;C:\\WINDOWS\\System32\\Wbem;C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\;C:\\WINDOWS\\System32\\OpenSSH\\;C:\\Users\\Tejas\\AppData\\Local\\Programs\\Python\\Python39;C:\\flutter_windows_3.13.3-stable\\flutter\\bin;C:\\Program Files\\Git\\cmd;C:\\MinGW\\bin;C:\\Program Files\\nodejs\\;C:\\ProgramData\\chocolatey\\bin;c:\\users\\tejas\\appdata\\roaming\\python\\python312\\site-packages;;C:\\Program Files\\Docker\\Docker\\resources\\bin;C:\\Program Files\\dotnet\\;C:\\Program Files\\MySQL\\MySQL Shell 8.0\\bin\\;C:\\Users\\Tejas\\AppData\\Local\\Microsoft\\WindowsApps;C:\\Users\\Tejas\\AppData\\Local\\GitHubDesktop\\bin;C:\\MinGW\\bin;C:\\iverilog\\bin;C:\\Users\\Tejas\\AppData\\Roaming\\npm;C:\\Users\\Tejas\\AppData\\Local\\Programs\\Microsoft VS Code\\bin;c:\\users\\tejas\\appdata\\roaming\\python\\python312\\site-packages;C:\\Users\\Tejas\\AppData\\Roaming\\Python\\Python312\\Scripts;C:\\GnuWin32\\bin;C:\\Program Files\\MySQL\\MySQL Server 8.0\\bin;C:\\Program Files\\Java\\jdk-25.0.2\\bin;C:\\Program Files (x86)\\apache-maven-3.9.13\\bin;C:\\Users\\Tejas\\AppData\\Local\\Programs\\Antigravity\\bin;C:\\Users\\Tejas\\AppData\\Local\\Microsoft\\WinGet\\Links;;c:\\Users\\Tejas\\.vscode\\extensions\\ms-python.debugpy-2026.6.0-win32-x64\\bundled\\scripts\\noConfigScripts","PATHEXT":".COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC;.PY;.PYW;.CPL","platformcode":"KV","PROCESSOR_ARCHITECTURE":"AMD64","PROCESSOR_IDENTIFIER":"AMD64 Family 23 Model 24 Stepping 1, AuthenticAMD","PROCESSOR_LEVEL":"23","PROCESSOR_REVISION":"1801","ProgramData":"C:\\ProgramData","ProgramFiles":"C:\\Program Files","ProgramFiles(x86)":"C:\\Program Files (x86)","ProgramW6432":"C:\\Program Files","PSModulePath":"D:\\OneDrive\\Documents\\WindowsPowerShell\\Modules;C:\\Program Files\\WindowsPowerShell\\Modules;C:\\WINDOWS\\system32\\WindowsPowerShell\\v1.0\\Modules","PUBLIC":"C:\\Users\\Public","

RegionCode":"APJ","SESSIONNAME":"Console","SystemDrive":"C:","SystemRoot":"C:\\WINDOWS","TEMP":"C:\\Users\\Tejas\\AppData\\Local\\Temp","TMP":"C:\\Users\\Tejas\\AppData\\Local\\Temp","USERDOMAIN":"LAPTOP-KGMNJSJJ","USERDOMAIN_ROAMINGPROFILE":"LAPTOP-KGMNJSJJ","USERNAME":"Tejas","USERPROFILE":"C:\\Users\\Tejas","VBOX_HWVIRTEX_IGNORE_SVM_IN_USE":"1","VBOX_MSI_INSTALL_PATH":"D:\\Virtual Box\\","VSCODE_PYTHON_AUTOACTIVATE_GUARD":"1","windir":"C:\\WINDOWS","TERM_PROGRAM":"vscode","TERM_PROGRAM_VERSION":"1.126.0","LANG":"en_US.UTF-8","COLORTERM":"truecolor","COPILOT_DEBUG_NONCE":"ae48a558418a6d20da24aef174965ad6","GIT_ASKPASS":"c:\\Users\\Tejas\\AppData\\Roaming\\Code\\User\\globalStorage\\vscode.git\\askpass\\70789581cae28aa7\\askpass.sh","VSCODE_GIT_ASKPASS_NODE":"C:\\Users\\Tejas\\AppData\\Local\\Programs\\Microsoft VS Code\\Code.exe","VSCODE_GIT_ASKPASS_EXTRA_ARGS":"","VSCODE_GIT_ASKPASS_MAIN":"c:\\Users\\Tejas\\AppData\\Roaming\\Code\\User\\globalStorage\\vscode.git\\askpass\\70789581cae28aa7\\askpass-main.js","VSCODE_GIT_IPC_HANDLE":"\\\\.\\pipe\\vscode-git-951db35517-sock","PYDEVD_DISABLE_FILE_VALIDATION":"1","VSCODE_DEBUGPY_ADAPTER_ENDPOINTS":"c:\\Users\\Tejas\\.vscode\\extensions\\ms-python.debugpy-2026.6.0-win32-x64\\.noConfigDebugAdapterEndpoints\\endpoint-545145662d2be203.txt","BUNDLED_DEBUGPY_PATH":"c:\\Users\\Tejas\\.vscode\\extensions\\ms-python.debugpy-2026.6.0-win32-x64\\bundled\\libs\\debugpy","PYTHONSTARTUP":"c:\\Users\\Tejas\\AppData\\Roaming\\Code\\User\\workspaceStorage\\b0b47d5384da2be4a187615101156066\\ms-python.python\\pythonrc.py","PYTHON_BASIC_REPL":"1","VSCODE_INJECTION":"1"}}.

Solution: Avoid exposing detailed error messages in production; use generic error responses.

Invalid Content-Type

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/prices (HTTP equivalent) serves invalid Content-Type: text/html.

Solution: Ensure WebSocket endpoints return appropriate Content-Type or upgrade headers.

Missing Security Headers

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/prices (HTTP equivalent) lacks the following headers: Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options.

Solution: Add missing security headers such as Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security.

Connection Flood

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices allowed 100 concurrent connections in 1.96s.

Solution: Enforce per-IP connection limits and rate limiting to prevent abuse.

Oversized Message

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted a 10MB message.

Solution: Set a reasonable max message size limit (e.g., 1MB) to prevent buffer overflows.

Max Connections

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices allows 100 simultaneous connections without restriction.

Solution: Enforce a maximum connection limit per client to prevent resource exhaustion.

Idle Timeout Abuse

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices allows idle connections to persist for 60 seconds.

Solution: Implement an idle timeout policy to close inactive connections.

High Compression Ratio

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepts highly compressible messages (1MB of 'A').

Solution: Limit allowed compression ratio or message size on the server.

Large Payload Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted repeated large messages without closing.

Solution: Set server-side limits for message size and rate. Monitor memory usage.

TCP Half-Open Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/prices accepted hanging TCP connections without timeout.

Solution: Use TCP keep-alive and server-side timeout policies.

No Compression Negotiation

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/prices may mishandle compression without proper negotiation.

Solution: Ensure the server only decompresses messages when permessage-deflate was negotiated.

Affected WebSocket Endpoint: ws://localhost:3000/ws/comments

Case-Sensitive Headers

Risk Level: Low

Description: Server at localhost:3000 accepted case-sensitive headers.

Solution: Ensure case-insensitive header parsing as per RFC.

Oversized Headers

Risk Level: Medium

Description: Server at localhost:3000 accepted handshake with oversized headers.

Solution: Set limits for header size to prevent resource exhaustion.

Missing Host Header

Risk Level: High

Description: Server at localhost:3000 accepted handshake without Host header.

Solution: Enforce Host header validation.

Fake Host Header

Risk Level: High

Description: Server at localhost:3000 accepted handshake with incorrect Host header.

Solution: Validate Host header to match expected server domain.

Multiple Host Headers

Risk Level: High

Description: Server at localhost:3000 accepted handshake with multiple Host headers.

Solution: Reject requests with duplicate Host headers.

No Session Cookie

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepts connections without a session cookie.

Solution: Require valid session cookies (or tokens) to authenticate WebSocket clients.

Expired Cookie

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments accepts connections with an expired session cookie.

Solution: Validate cookie expiration on the server side and reject expired tokens.

Fake Token

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepts connections with a fake authentication token.

Solution: Implement robust token validation (e.g., JWT signature verification, token expiry check, audience validation).

Stale Session Reconnect

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments allows reconnection with same stale session cookie.

Solution: Invalidate old session IDs on WebSocket reconnect. Require fresh authentication or refresh token.

Cross-Site Cookie Hijack

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted cross-origin cookies and origin header.

Solution: Set SameSite=Strict on cookies and validate the Origin header server-side.

Missing Authentication

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments allows unauthenticated connections and responds with data.

Solution: Require authentication (e.g., JWT, API keys) for WebSocket connections.

Invalid Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments negotiated invalid subprotocol: 'invalid..protocol'.

Solution: Reject malformed or unsupported subprotocol values during handshake.

Unaccepted Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments negotiated unadvertised subprotocol 'unadvertised_protocol'.

Solution: Only negotiate subprotocols explicitly supported by the server.

Fake Extension

Risk Level: High

Description: Server at localhost:3000 accepted spoofed extension.

Solution: Validate Sec-WebSocket-Extensions header against supported values.

Conflicting Extensions

Risk Level: Medium

Description: Server at localhost:3000 accepted conflicting extensions.

Solution: Reject requests with duplicate or conflicting extensions.

Spoofed Connection Header

Risk Level: High

Description: Server at localhost:3000 accepted spoofed Connection header.

Solution: Strictly validate Connection header to be exactly "Upgrade".

HTTP/1.0 Downgrade

Risk Level: High

Description: Server at localhost:3000 accepted HTTP/1.0 WebSocket handshake.

Solution: Only allow WebSocket upgrades over HTTP/1.1 or newer.

Undefined Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted frame with undefined opcode 0x3.

Solution: Reject frames with undefined opcodes.

Reserved Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted frame with reserved opcode 0xB.

Solution: Reject frames with reserved opcodes (0x3-0x7, 0xB-0xF).

Zero-Length Fragment

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/comments accepted zero-length fragments and responded unexpectedly.

Solution: Reject or limit incomplete fragmented messages.

Invalid Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted frame with declared payload length 10 but sent only 4 bytes.

Solution: Validate payload length matches actual data.

Negative Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted forged extended payload length (0x8000000000000001).

Solution: Validate payload length fields and reject extreme or invalid values.

Mismatched Payload

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments accepted frames with mismatched lengths.

Solution: Ensure payload lengths match.

Invalid Masking Key

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted a frame with invalid masking key pattern: All-zero.

Solution: Enforce strict validation of client masking keys per RFC 6455.

Unmasked Client Frame

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted an unmasked client frame.

Solution: Require masking for all client-to-server frames per RFC 6455.

Invalid RSV Bits

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments accepted a frame with invalid RSV1 bit set.

Solution: Reject non-zero RSV bits unless explicitly negotiated via extension.

Oversized Control Frame

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments accepted a ping control frame with 126-byte payload.

Solution: Reject control frames larger than 125 bytes as per RFC 6455.

Non-UTF-8 Text

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted a text frame with invalid UTF-8 bytes.

Solution: Ensure strict UTF-8 validation of text frames.

Null Bytes in Text

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments accepted a text frame containing null bytes.

Solution: Validate and sanitize text frames for embedded nulls. Avoid C-style string truncation risks.

Binary as Text

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/comments accepted a text frame with non-UTF-8 binary data.

Solution: Validate UTF-8 compliance in all text frames as per RFC 6455.

Text as Binary

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/comments accepted UTF-8 text sent in a binary frame.

Solution: Handle binary and text frames with separate logic as per RFC 6455.

Invalid Close Code

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments accepted a close frame with invalid code 999.

Solution: Close codes must conform to RFC 6455 (valid: 1000-1015, 3000-4999).

Early Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/comments accepted an early close frame before any data was exchanged.

Solution: Gracefully handle close frames sent immediately after handshake.

No Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/comments handled abrupt TCP closure and allowed clean reconnection.

Solution: Ensure that server detects and cleans up on ungraceful disconnects.

Long Close Reason

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments accepted close frame with long reason (123 bytes).

Solution: Enforce strict limits on close reason size (≤123 bytes).

Missing CORS Headers

Risk Level: High

Description: WebSocket endpoint ws://localhost:3000/ws/comments (HTTP equivalent) lacks proper CORS headers.

Solution: Implement proper CORS headers to restrict cross-origin access.

Cross-Origin Iframe

Risk Level: High

Description: ws://localhost:3000/ws/comments allows itself to be embedded in cross-origin iframes (missing X-Frame-Options / CSP).

Solution: Set X-Frame-Options: DENY or SAMEORIGIN, or CSP frame-ancestors directive.

Missing Origin Check

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepts connections from unauthorized origin 'http://malicious-site.com'.

Solution: Implement strict Origin header validation (whitelist allowed domains).

Invalid Content-Type

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/comments (HTTP equivalent) serves invalid Content-Type: text/html.

Solution: Ensure WebSocket endpoints return appropriate Content-Type or upgrade headers.

Missing Security Headers

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/comments (HTTP equivalent) lacks the following headers: Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options.

Solution: Add missing security headers such as Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security.

Connection Flood

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments allowed 100 concurrent connections in 0.78s.

Solution: Enforce per-IP connection limits and rate limiting to prevent abuse.

Oversized Message

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted a 10MB message.

Solution: Set a reasonable max message size limit (e.g., 1MB) to prevent buffer overflows.

Max Connections

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments allows 100 simultaneous connections without restriction.

Solution: Enforce a maximum connection limit per client to prevent resource exhaustion.

Idle Timeout Abuse

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments allows idle connections to persist for 60 seconds.

Solution: Implement an idle timeout policy to close inactive connections.

High Compression Ratio

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepts highly compressible messages (1MB of 'A').

Solution: Limit allowed compression ratio or message size on the server.

Large Payload Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted repeated large messages without closing.

Solution: Set server-side limits for message size and rate. Monitor memory usage.

TCP Half-Open Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/comments accepted hanging TCP connections without timeout.

Solution: Use TCP keep-alive and server-side timeout policies.

No Compression Negotiation

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/comments may mishandle compression without proper negotiation.

Solution: Ensure the server only decompresses messages when permessage-deflate was negotiated.

Affected WebSocket Endpoint: ws://localhost:3000/ws/notify

Case-Sensitive Headers

Risk Level: Low

Description: Server at localhost:3000 accepted case-sensitive headers.

Solution: Ensure case-insensitive header parsing as per RFC.

Oversized Headers

Risk Level: Medium

Description: Server at localhost:3000 accepted handshake with oversized headers.

Solution: Set limits for header size to prevent resource exhaustion.

Missing Host Header

Risk Level: High

Description: Server at localhost:3000 accepted handshake without Host header.

Solution: Enforce Host header validation.

Fake Host Header

Risk Level: High

Description: Server at localhost:3000 accepted handshake with incorrect Host header.

Solution: Validate Host header to match expected server domain.

Multiple Host Headers

Risk Level: High

Description: Server at localhost:3000 accepted handshake with multiple Host headers.

Solution: Reject requests with duplicate Host headers.

No Session Cookie

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepts connections without a session cookie.

Solution: Require valid session cookies (or tokens) to authenticate WebSocket clients.

Expired Cookie

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify accepts connections with an expired session cookie.

Solution: Validate cookie expiration on the server side and reject expired tokens.

Fake Token

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepts connections with a fake authentication token.

Solution: Implement robust token validation (e.g., JWT signature verification, token expiry check, audience validation).

Stale Session Reconnect

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify allows reconnection with same stale session cookie.

Solution: Invalidate old session IDs on WebSocket reconnect. Require fresh authentication or refresh token.

Cross-Site Cookie Hijack

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted cross-origin cookies and origin header.

Solution: Set SameSite=Strict on cookies and validate the Origin header server-side.

Missing Authentication

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify allows unauthenticated connections and responds with data.

Solution: Require authentication (e.g., JWT, API keys) for WebSocket connections.

Invalid Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify negotiated invalid subprotocol: 'invalid..protocol'.

Solution: Reject malformed or unsupported subprotocol values during handshake.

Unaccepted Subprotocol

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify negotiated unadvertised subprotocol 'unadvertised_protocol'.

Solution: Only negotiate subprotocols explicitly supported by the server.

Fake Extension

Risk Level: High

Description: Server at localhost:3000 accepted spoofed extension.

Solution: Validate Sec-WebSocket-Extensions header against supported values.

Conflicting Extensions

Risk Level: Medium

Description: Server at localhost:3000 accepted conflicting extensions.

Solution: Reject requests with duplicate or conflicting extensions.

Spoofed Connection Header

Risk Level: High

Description: Server at localhost:3000 accepted spoofed Connection header.

Solution: Strictly validate Connection header to be exactly "Upgrade".

HTTP/1.0 Downgrade

Risk Level: High

Description: Server at localhost:3000 accepted HTTP/1.0 WebSocket handshake.

Solution: Only allow WebSocket upgrades over HTTP/1.1 or newer.

Undefined Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted frame with undefined opcode 0x3.

Solution: Reject frames with undefined opcodes.

Reserved Opcode

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted frame with reserved opcode 0xB.

Solution: Reject frames with reserved opcodes (0x3-0x7, 0xB-0xF).

Zero-Length Fragment

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/notify accepted zero-length fragments and responded unexpectedly.

Solution: Reject or limit incomplete fragmented messages.

Invalid Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted frame with declared payload length 10 but sent only 4 bytes.

Solution: Validate payload length matches actual data.

Negative Payload Length

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted forged extended payload length (0x8000000000000001).

Solution: Validate payload length fields and reject extreme or invalid values.

Mismatched Payload

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify accepted frames with mismatched lengths.

Solution: Ensure payload lengths match.

Invalid Masking Key

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted a frame with invalid masking key pattern: All-zero.

Solution: Enforce strict validation of client masking keys per RFC 6455.

Unmasked Client Frame

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted an unmasked client frame.

Solution: Require masking for all client-to-server frames per RFC 6455.

Invalid RSV Bits

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify accepted a frame with invalid RSV1 bit set.

Solution: Reject non-zero RSV bits unless explicitly negotiated via extension.

Oversized Control Frame

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify accepted a ping control frame with 126-byte payload.

Solution: Reject control frames larger than 125 bytes as per RFC 6455.

Non-UTF-8 Text

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted a text frame with invalid UTF-8 bytes.

Solution: Ensure strict UTF-8 validation of text frames.

Null Bytes in Text

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify accepted a text frame containing null bytes.

Solution: Validate and sanitize text frames for embedded nulls. Avoid C-style string truncation risks.

Binary as Text

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/notify accepted a text frame with non-UTF-8 binary data.

Solution: Validate UTF-8 compliance in all text frames as per RFC 6455.

Text as Binary

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/notify accepted UTF-8 text sent in a binary frame.

Solution: Handle binary and text frames with separate logic as per RFC 6455.

Invalid Close Code

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify accepted a close frame with invalid code 999.

Solution: Close codes must conform to RFC 6455 (valid: 1000-1015, 3000-4999).

Early Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/notify accepted an early close frame before any data was exchanged.

Solution: Gracefully handle close frames sent immediately after handshake.

No Close Frame

Risk Level: Low

Description: WebSocket at ws://localhost:3000/ws/notify handled abrupt TCP closure and allowed clean reconnection.

Solution: Ensure that server detects and cleans up on ungraceful disconnects.

Long Close Reason

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify accepted close frame with long reason (123 bytes).

Solution: Enforce strict limits on close reason size (≤123 bytes).

Missing CORS Headers

Risk Level: High

Description: WebSocket endpoint ws://localhost:3000/ws/notify (HTTP equivalent) lacks proper CORS headers.

Solution: Implement proper CORS headers to restrict cross-origin access.

Cross-Origin Iframe

Risk Level: High

Description: ws://localhost:3000/ws/notify allows itself to be embedded in cross-origin iframes (missing X-Frame-Options / CSP).

Solution: Set X-Frame-Options: DENY or SAMEORIGIN, or CSP frame-ancestors directive.

Missing Origin Check

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepts connections from unauthorized origin 'http://malicious-site.com'.

Solution: Implement strict Origin header validation (whitelist allowed domains).

Invalid Content-Type

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/notify (HTTP equivalent) serves invalid Content-Type: text/html.

Solution: Ensure WebSocket endpoints return appropriate Content-Type or upgrade headers.

Missing Security Headers

Risk Level: Medium

Description: WebSocket endpoint ws://localhost:3000/ws/notify (HTTP equivalent) lacks the following headers: Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options.

Solution: Add missing security headers such as Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security.

Connection Flood

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify allowed 100 concurrent connections in 0.98s.

Solution: Enforce per-IP connection limits and rate limiting to prevent abuse.

Oversized Message

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted a 10MB message.

Solution: Set a reasonable max message size limit (e.g., 1MB) to prevent buffer overflows.

Max Connections

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify allows 100 simultaneous connections without restriction.

Solution: Enforce a maximum connection limit per client to prevent resource exhaustion.

Idle Timeout Abuse

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify allows idle connections to persist for 60 seconds.

Solution: Implement an idle timeout policy to close inactive connections.

High Compression Ratio

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepts highly compressible messages (1MB of 'A').

Solution: Limit allowed compression ratio or message size on the server.

Large Payload Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted repeated large messages without closing.

Solution: Set server-side limits for message size and rate. Monitor memory usage.

TCP Half-Open Resource Leak

Risk Level: High

Description: WebSocket at ws://localhost:3000/ws/notify accepted hanging TCP connections without timeout.

Solution: Use TCP keep-alive and server-side timeout policies.

No Compression Negotiation

Risk Level: Medium

Description: WebSocket at ws://localhost:3000/ws/notify may mishandle compression without proper negotiation.

Solution: Ensure the server only decompresses messages when permessage-deflate was negotiated.